

Detecting and Describing



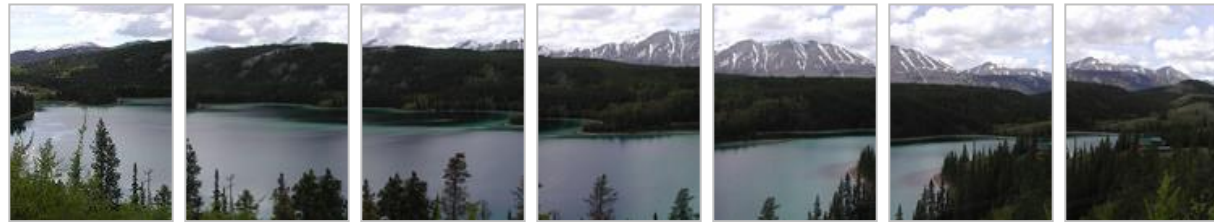
Dr. Praisan Padungweang, Collage of Computing, KKU

How to create a panorama



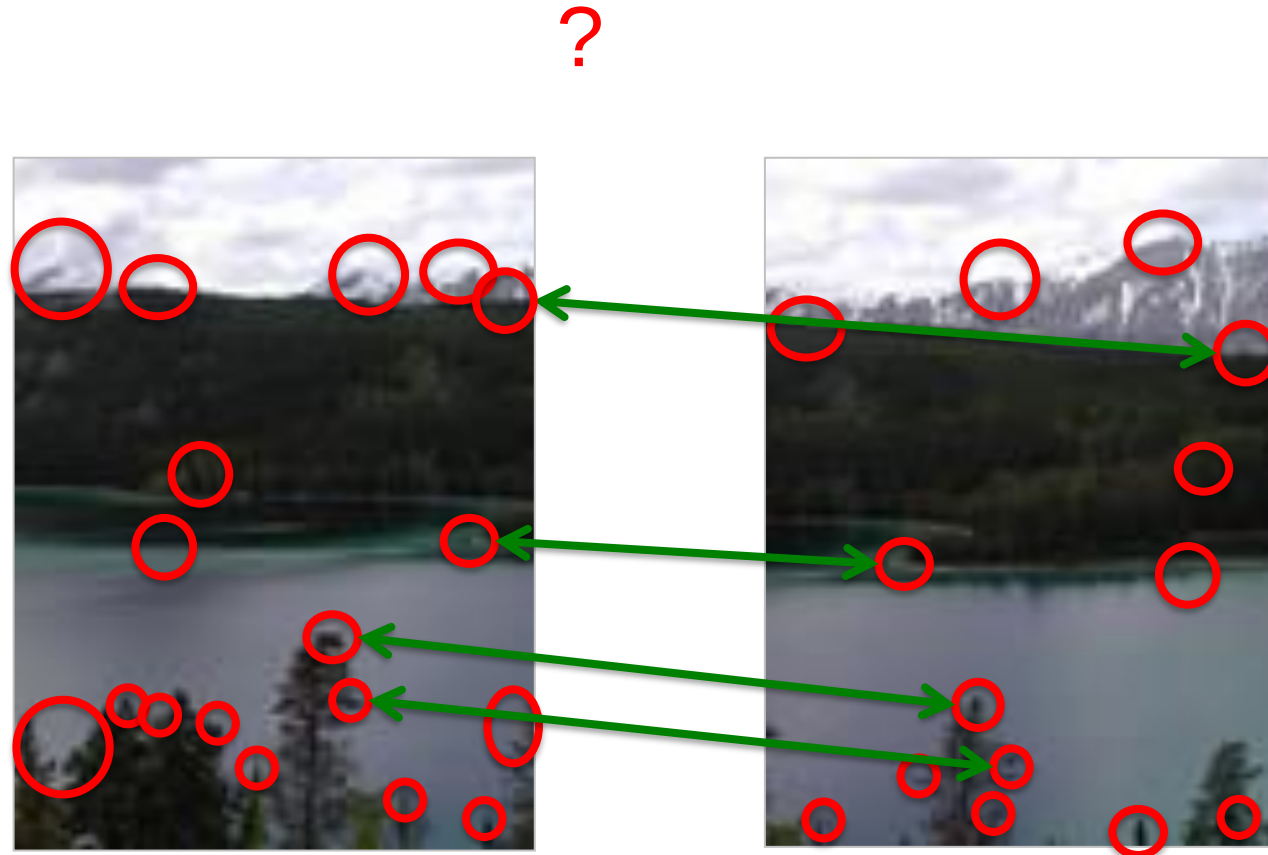
1. Take a sequence of images from the same position

- Rotate the camera about its optical center



2. Compute transformation between images

- Extract interest points
- Find Matches
- Compute transformation



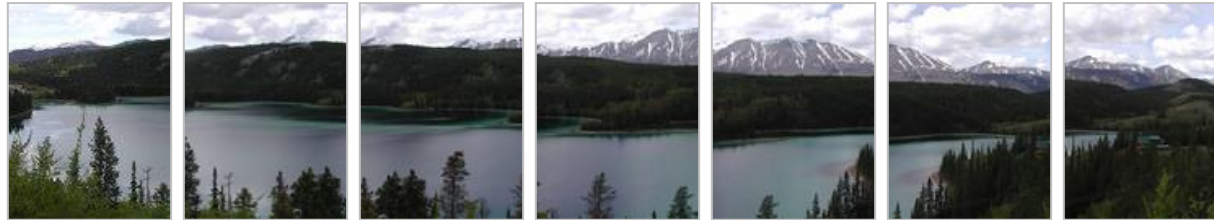
3. Shift the images to overlap



4. Blend the two together to create a mosaic



5. Repeat for all images



How to create a panorama

- Say we are stitching a panorama
- Want patches in image to match to other image
- Hopefully only match one spot

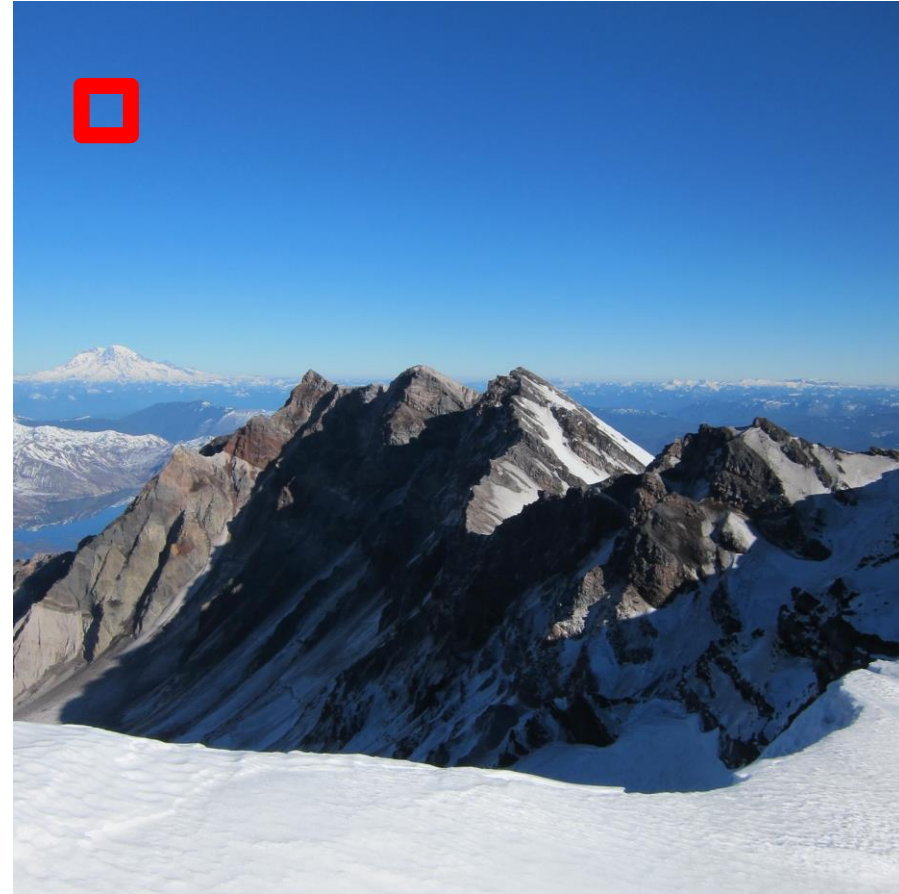


Q1: How close are two patches?

- Sum squared difference
- Images I, J
- $\sum_{x,y} (I(x,y) - J(x,y))^2$

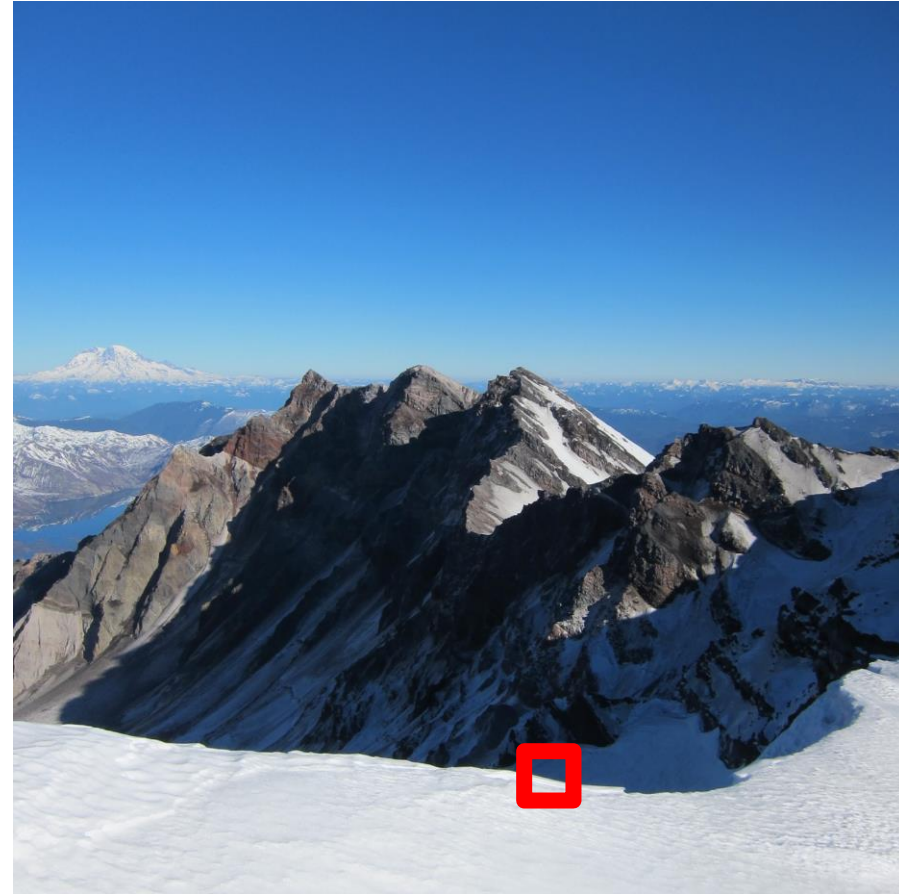
Q2: How can we find unique patches?

- Sky: bad
 - Very little variation
 - Could match any other sky



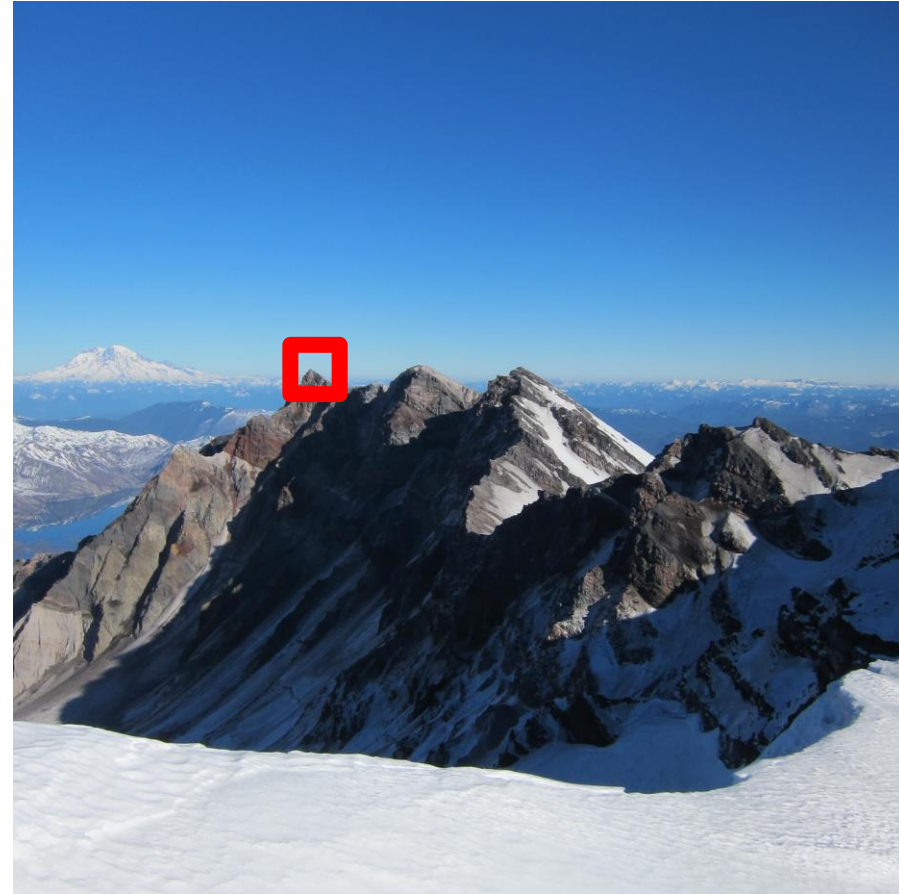
How can we find unique patches?

- Sky: bad
 - Very little variation
 - Could match any other sky
- Edge: ok
 - Variation in one direction
 - Could match other patches along same edge

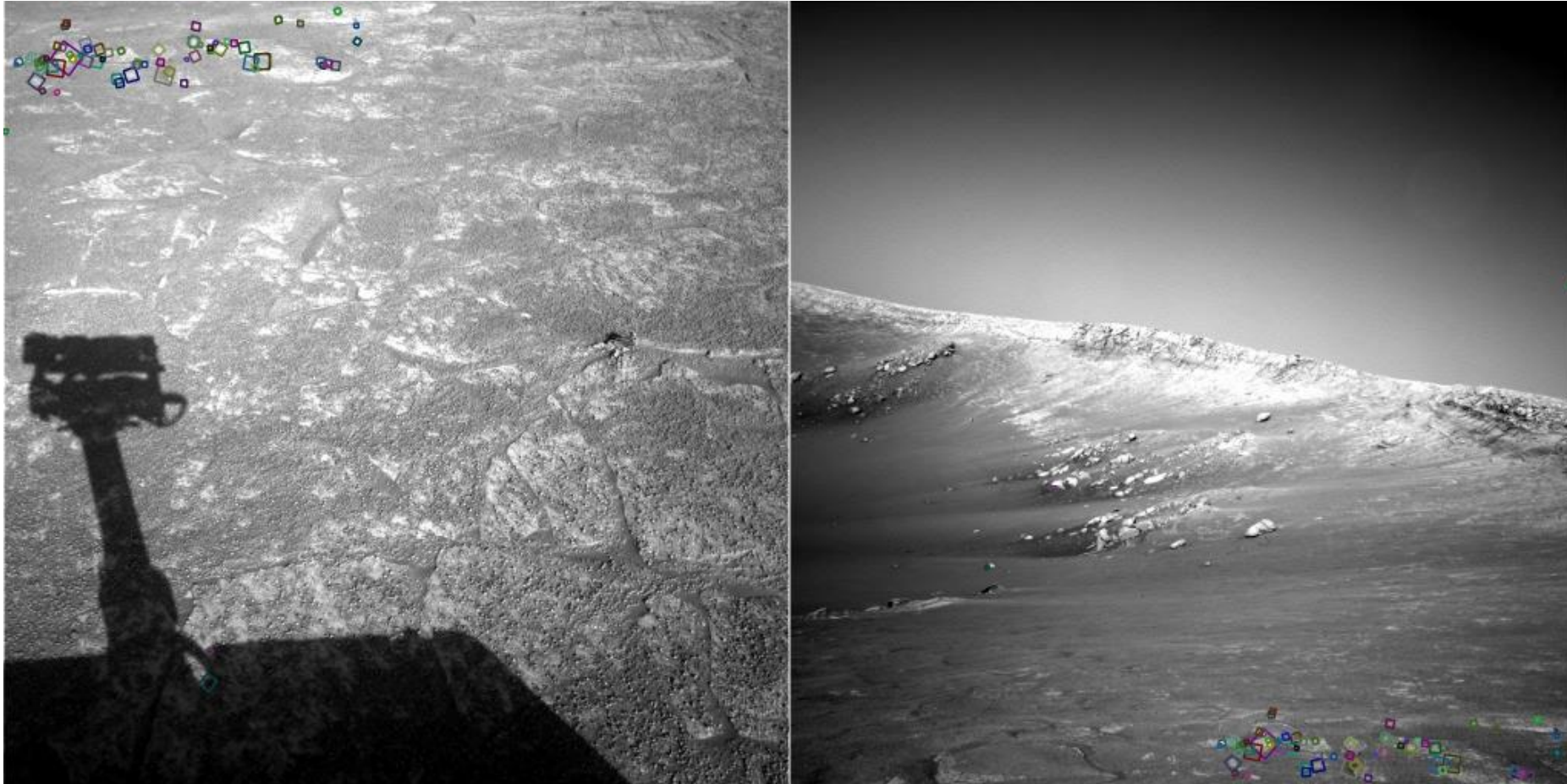


How can we find unique patches?

- Sky: bad
 - Very little variation
 - Could match any other sky
- Edge: ok
 - Variation in one direction
 - Could match other patches along same edge
- Corners: good!
 - Only one alignment matches



How many images?



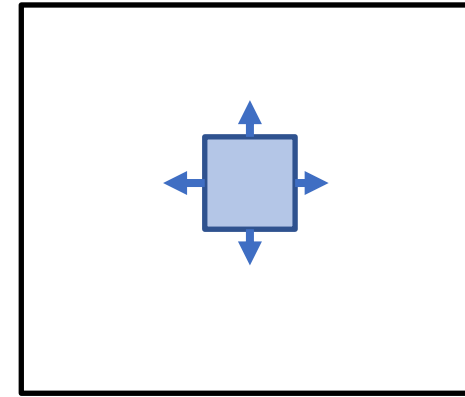
NASA Mars Rover images
with SIFT feature matches
Figure by Noah Snaveley

Interest Point Detection

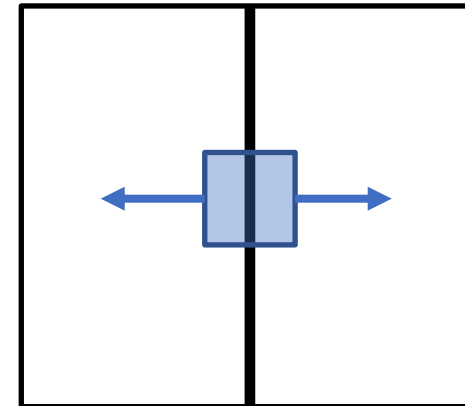
Interest Point Detection

Interest points have more complex gradients than edges

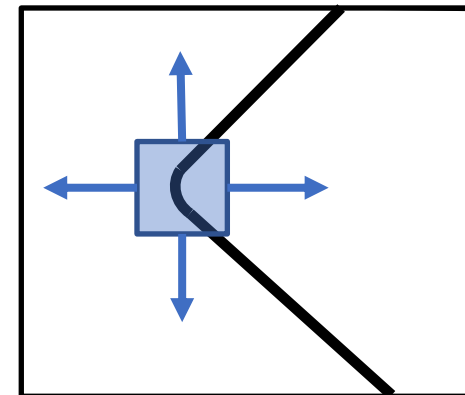
- Use convolutional kernel to estimate gradients
- Areas of similar intensity
- Edges have simple gradients perpendicular to the edges
- Other interest points have rapidly changing gradients in several directions



Similar intensity: Small gradient in all directions



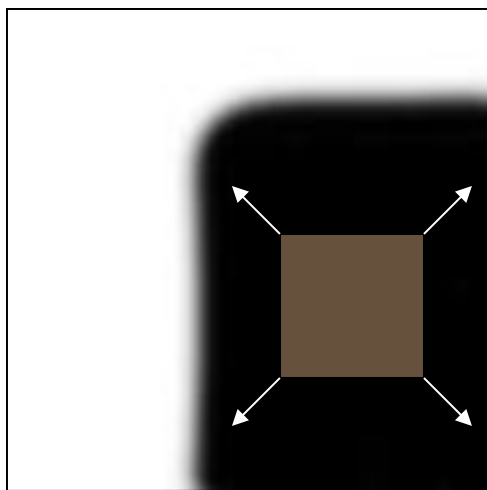
Edge: Large perpendicular gradients



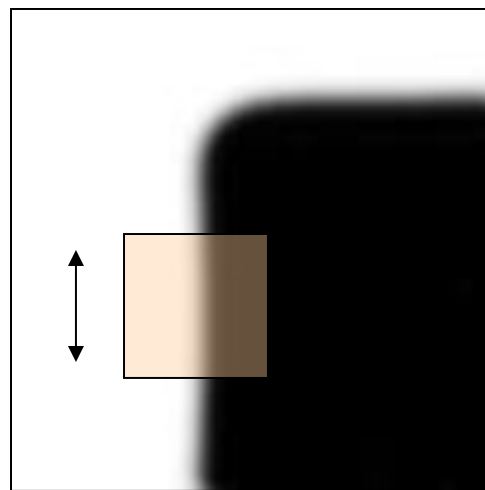
Interest point: Large gradient in all directions

Corners

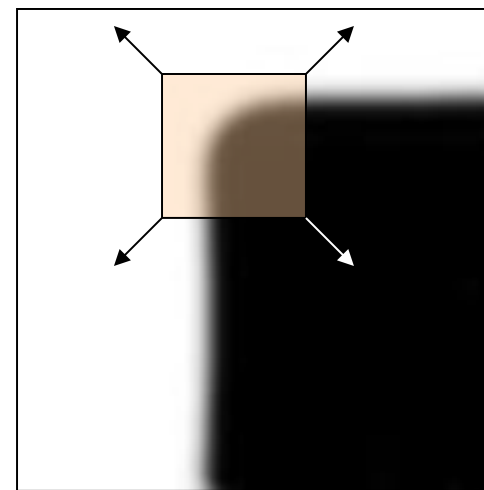
- We should easily recognize the point by looking through a small window
- Shifting a window in *any direction* should give a *large change* in intensity



“flat” region:
no change in
all directions

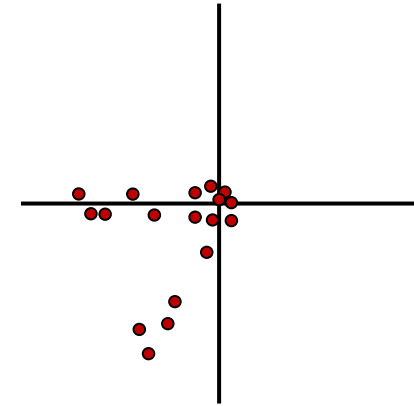
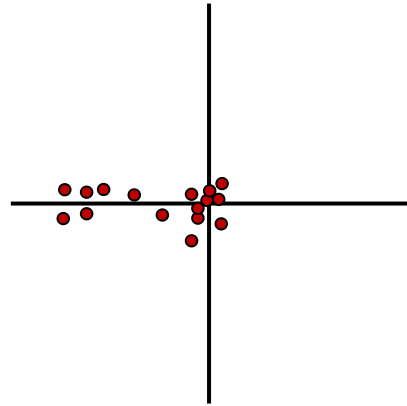
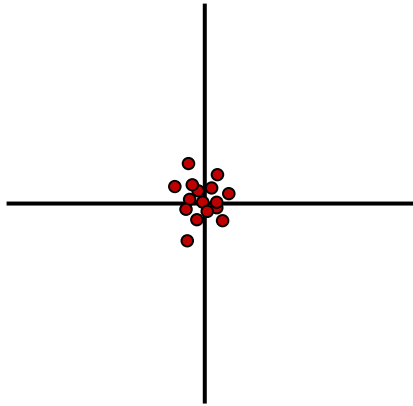
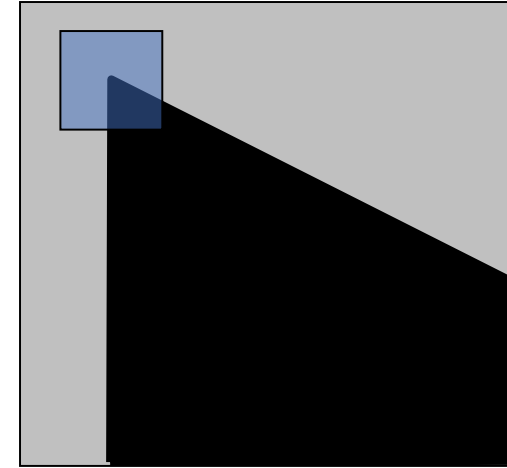
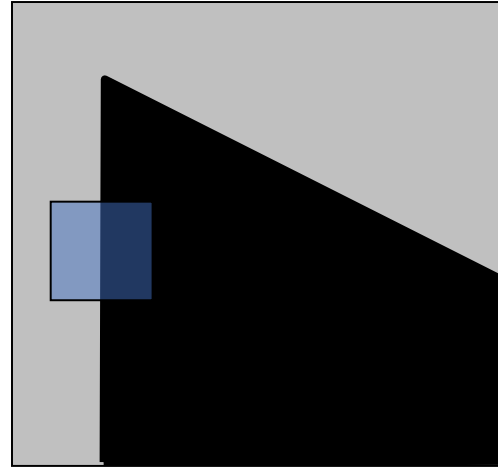
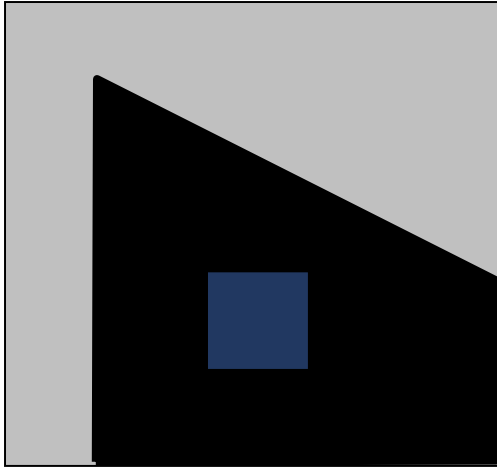


“edge”:
no change along
the edge
direction



“corner”:
significant
change in all
directions

Let's look at the **gradient** distributions



Principal Component Analysis

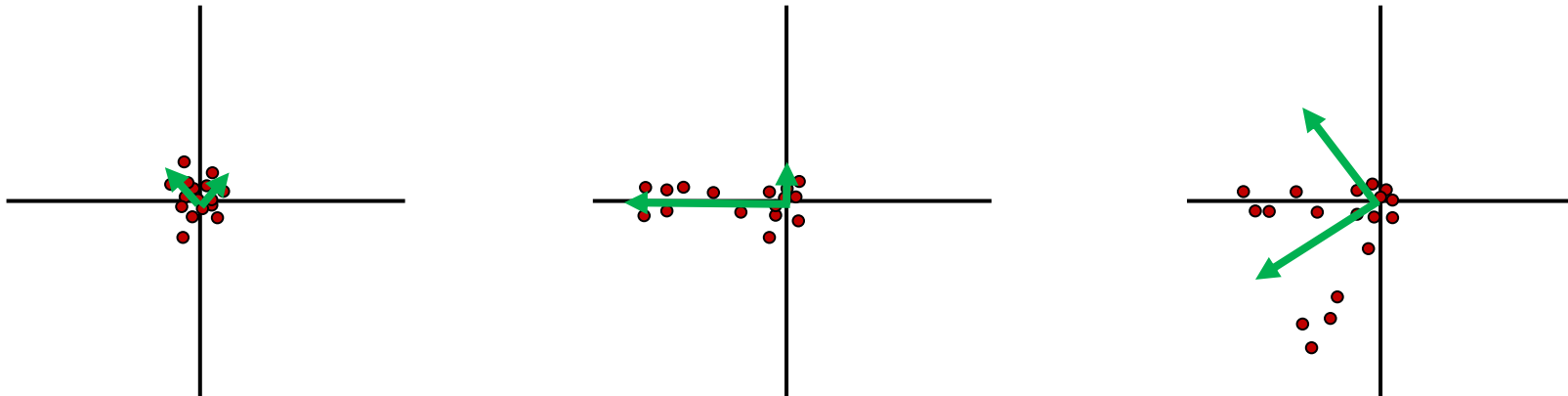
Principal component is the direction of highest variance.

Next, highest component is the direction with highest variance *orthogonal* to the previous components.

How to compute PCA components:

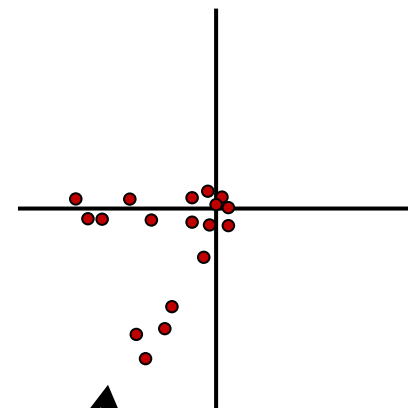
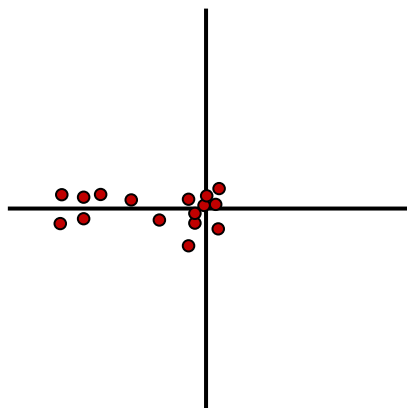
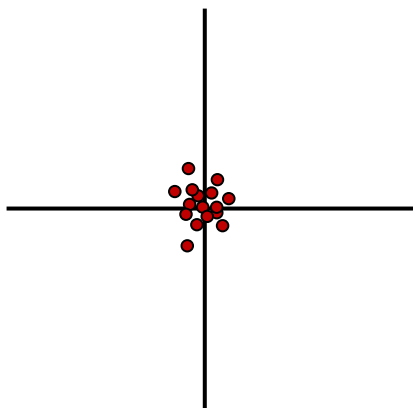
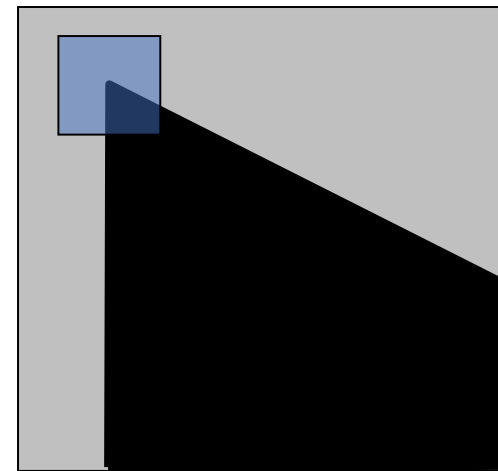
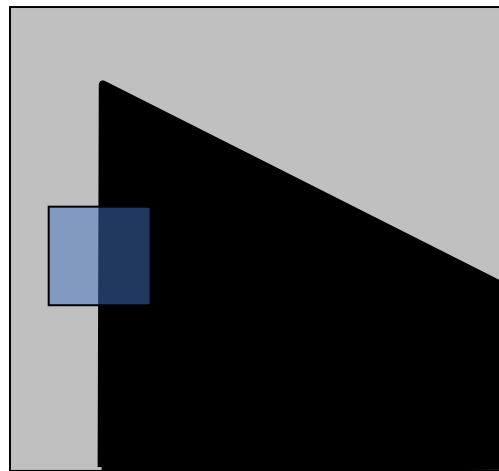
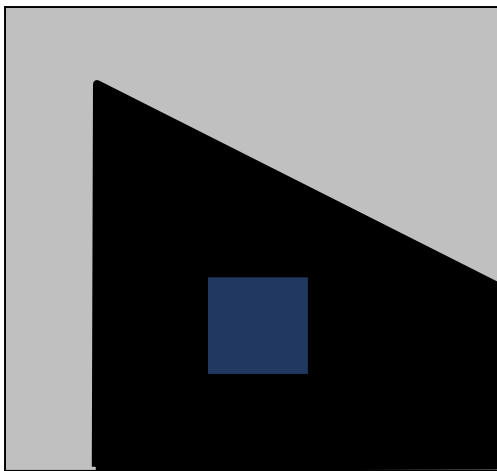
1. Subtract off the mean for each data point.
2. Compute the covariance matrix.
3. Compute eigenvectors and eigenvalues.
4. The components are the eigenvectors ranked by the eigenvalues.

$$Hx = \lambda x$$



Definition: A scalar λ is called an eigenvalue of the $n \times n$ matrix A if there is a nontrivial solution x of $Ax = \lambda x$. Such x is called an eigenvector corresponding to the eigenvalue λ .

Corners have ...

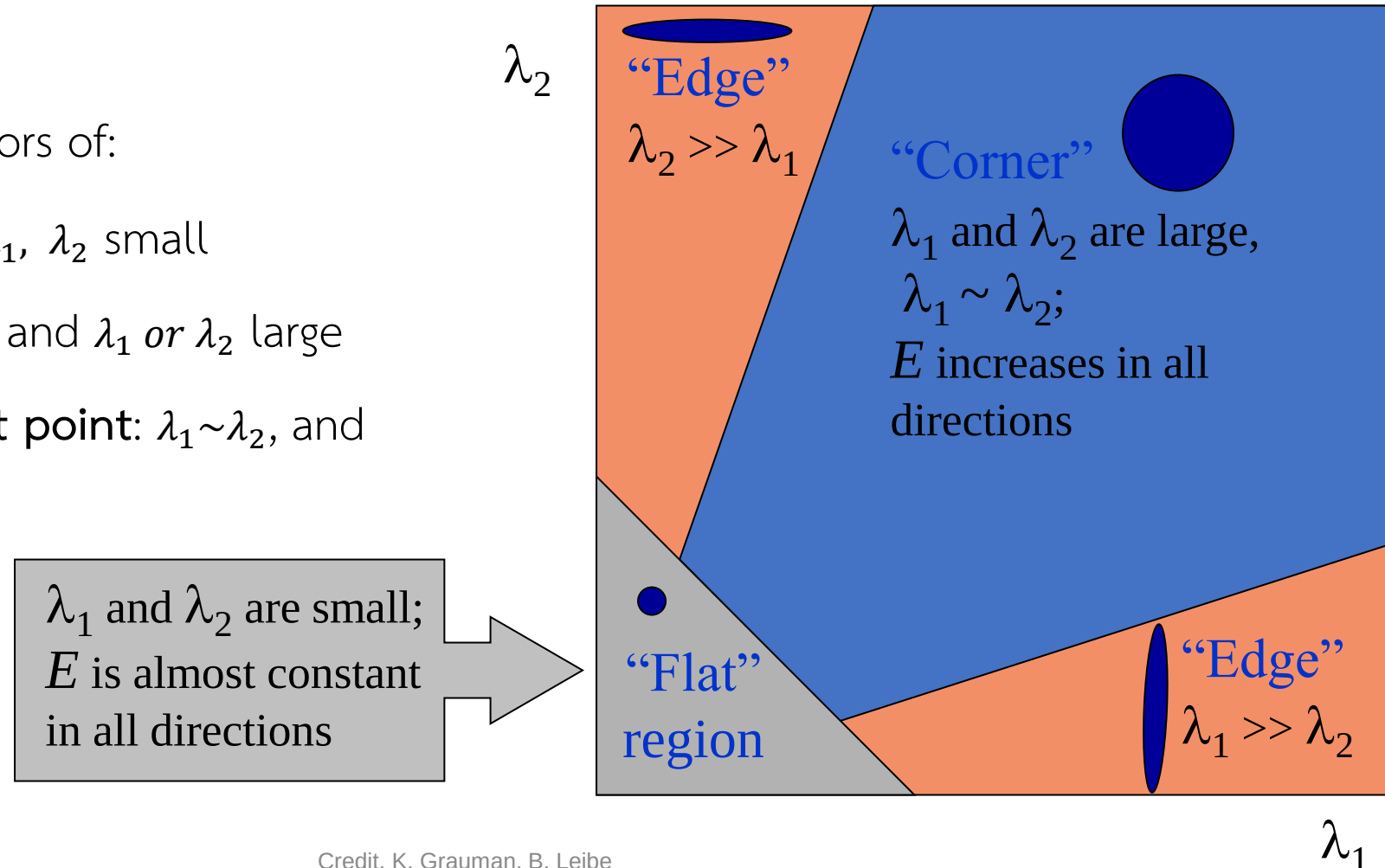


Both eigenvalues are large!

Interest Point Detection

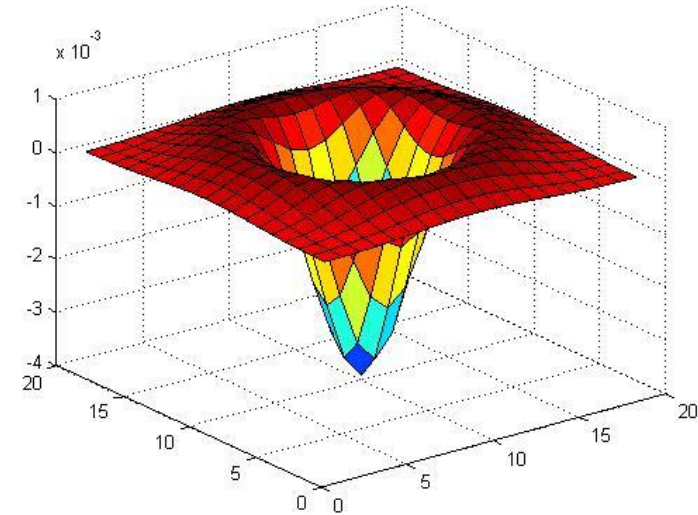
Interest point detection approach

- Eigenvalues of the Hessian, $[\lambda_1, \lambda_2]$, characterize the curvature
- Values of $[\lambda_1, \lambda_2]$ are indicators of:
 - Flat region: $\lambda_1 \sim \lambda_2$, and λ_1, λ_2 small
 - Edge: $\lambda_1 \gg \lambda_2$ or $\lambda_1 \ll \lambda_2$, and λ_1 or λ_2 large
 - Corner or other interest point: $\lambda_1 \sim \lambda_2$, and λ_1, λ_2 large



Laplacian of Gaussian

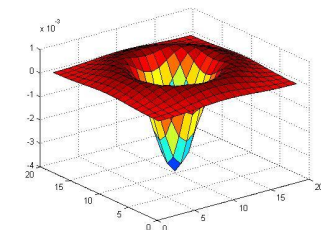
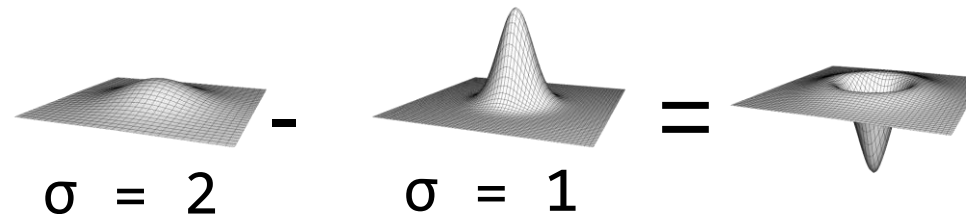
- use gaussian smoothing
- Can just use one kernel since convs commute
- Laplacian of Gaussian, LoG
- Can get good approx. with
5x5 - 9x9 kernels



Difference of Gaussian (DoG)

- Gaussian is a low pass filter
- Strongly reduce components with frequency $f < \sigma$
- $(g * I)$ low frequency components
- $I - (g * I)$ high frequency components
- $g(\sigma_1) * I - g(\sigma_2) * I$
 - Components in between these frequencies
- $g(\sigma_1) * I - g(\sigma_2) * I = [g(\sigma_1) - g(\sigma_2)] * I$

Laplacian of
Gaussian, LoG



Harris Algorithm

Harris Interest Point Detection

Harris interest point detection algorithm

- For efficiency, the Harris interest point detector avoids directly computing the eigenvalues

- Two properties of symmetric matrices:

$$\begin{aligned}\det(M) &= \lambda_1 \lambda_2 \\ \text{trace}(M) &= \lambda_1 + \lambda_2\end{aligned}$$

- The threshold, R , is then applied to the following quantity to find interest points

$$R < \det(M) + \alpha(\text{trace}(M))^2 = \lambda_1 \lambda_2 + \alpha(\lambda_1 + \lambda_2)^2$$

- Typical values $0.04 \leq \alpha \leq 0.06$
- Apply **non-maximal suppression**

Harris Interest Point Detection

Harris interest point detection algorithm

1. Adjust, filter and convert image to gray-scale
2. Using a Gaussian weighted window, $g(\sigma)$, compute the matrix M for each (overlapping) patch of the image
3. Apply the threshold function of each patch – non-maximal suppression by selecting the points with largest value of $\det(M) + \alpha(\text{trace}(M))^2$ in local regions

Harris Interest Point Detection

Steps of Harris interest point detection algorithm

1. Adjust, filter and convert image to gray-scale



Image from slide
by S. Lazebnik

Harris Interest Point Detection

Steps of Harris interest point detection algorithm

2. Using a Gaussian weighted window, $g(\sigma)$, compute the matrix M for each (overlapping) patch of the image

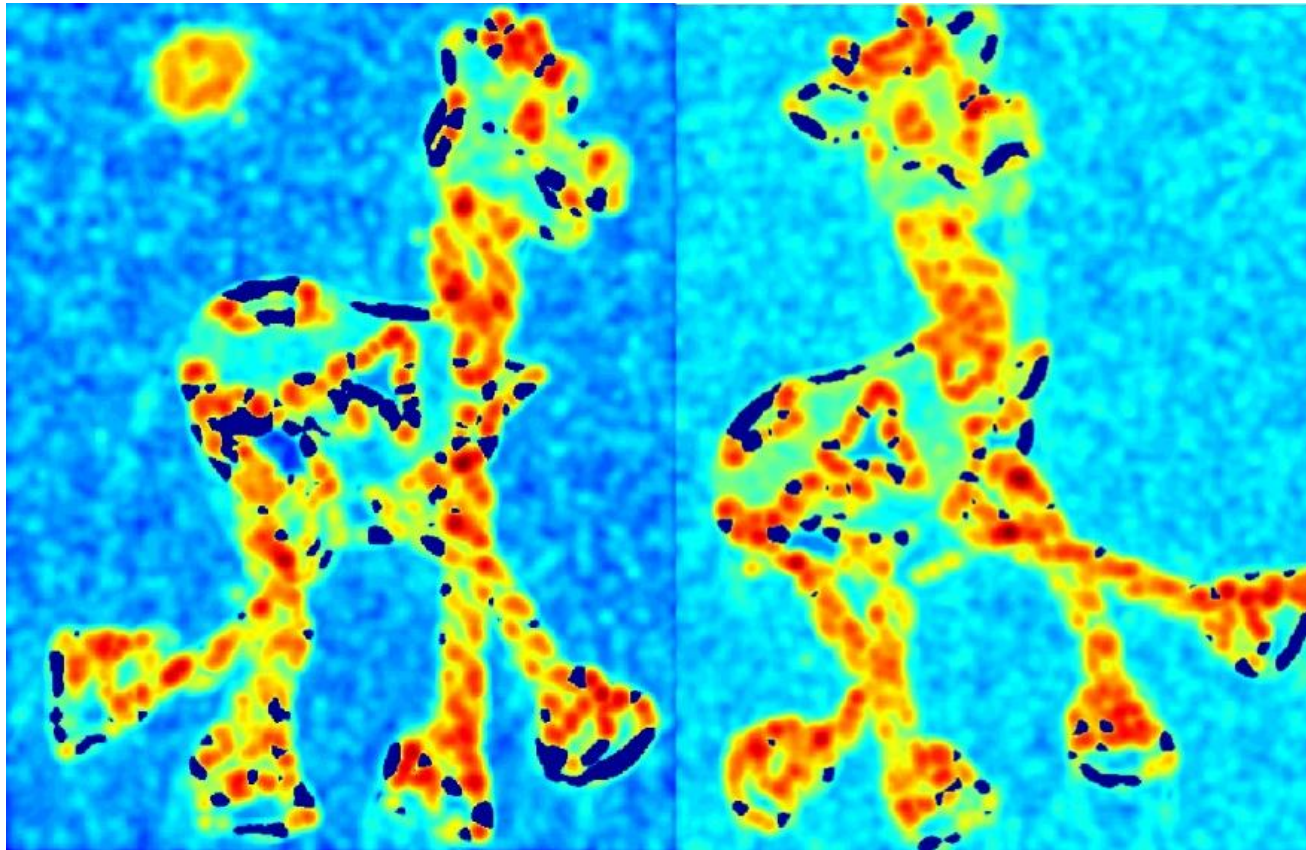


Image from slide
by S. Lazebnik

Harris Interest Point Detection

Steps of Harris interest point detection algorithm

3. Apply the threshold function of each patch

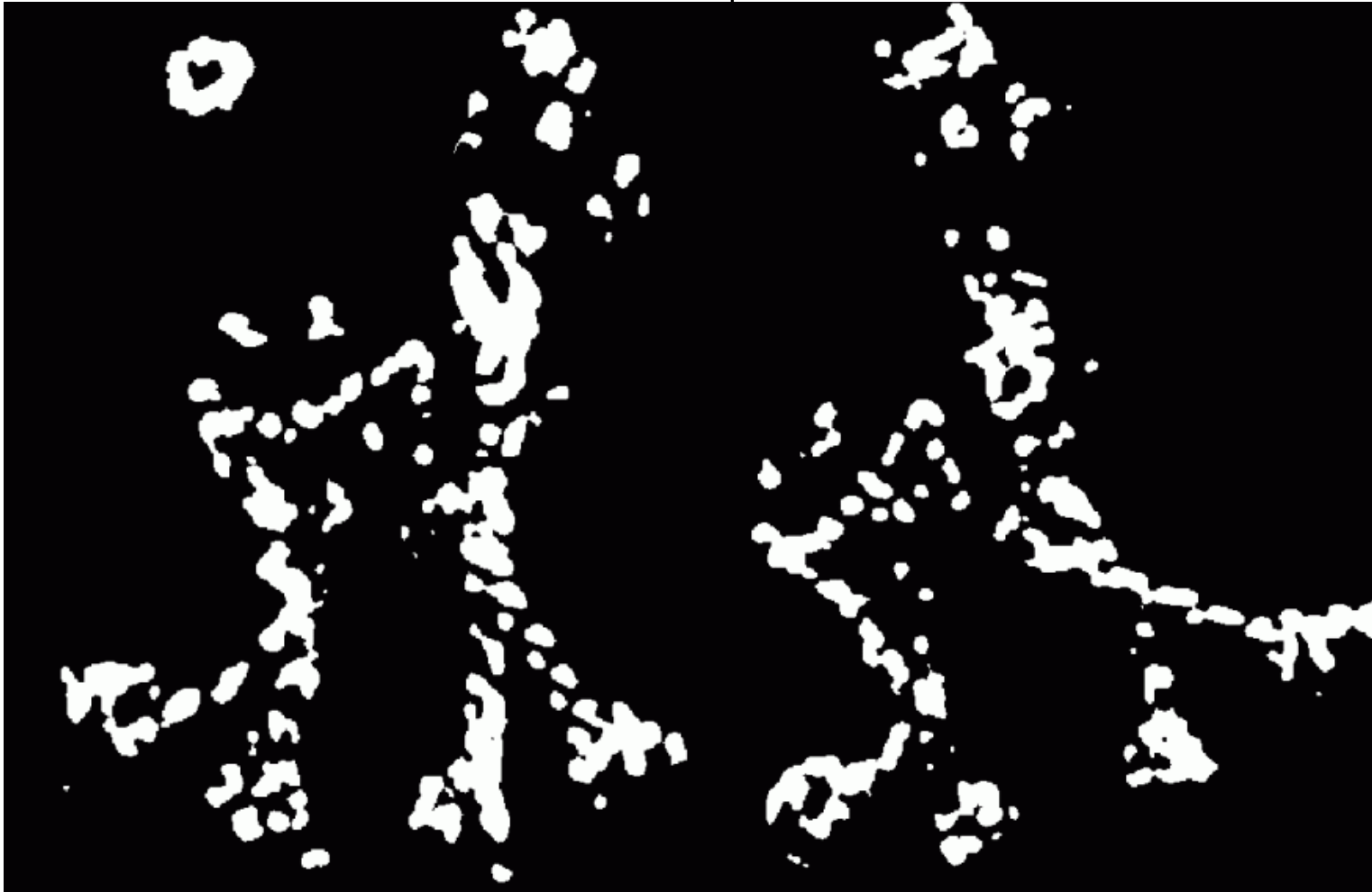


Image from slide
by S. Lazebnik

Harris Interest Point Detection

Steps of Harris interest point detection algorithm

4. Perform non-maximal suppression, by selecting the points with largest value of $\det(M) + \alpha(\text{trace}(M))^2$ in local regions

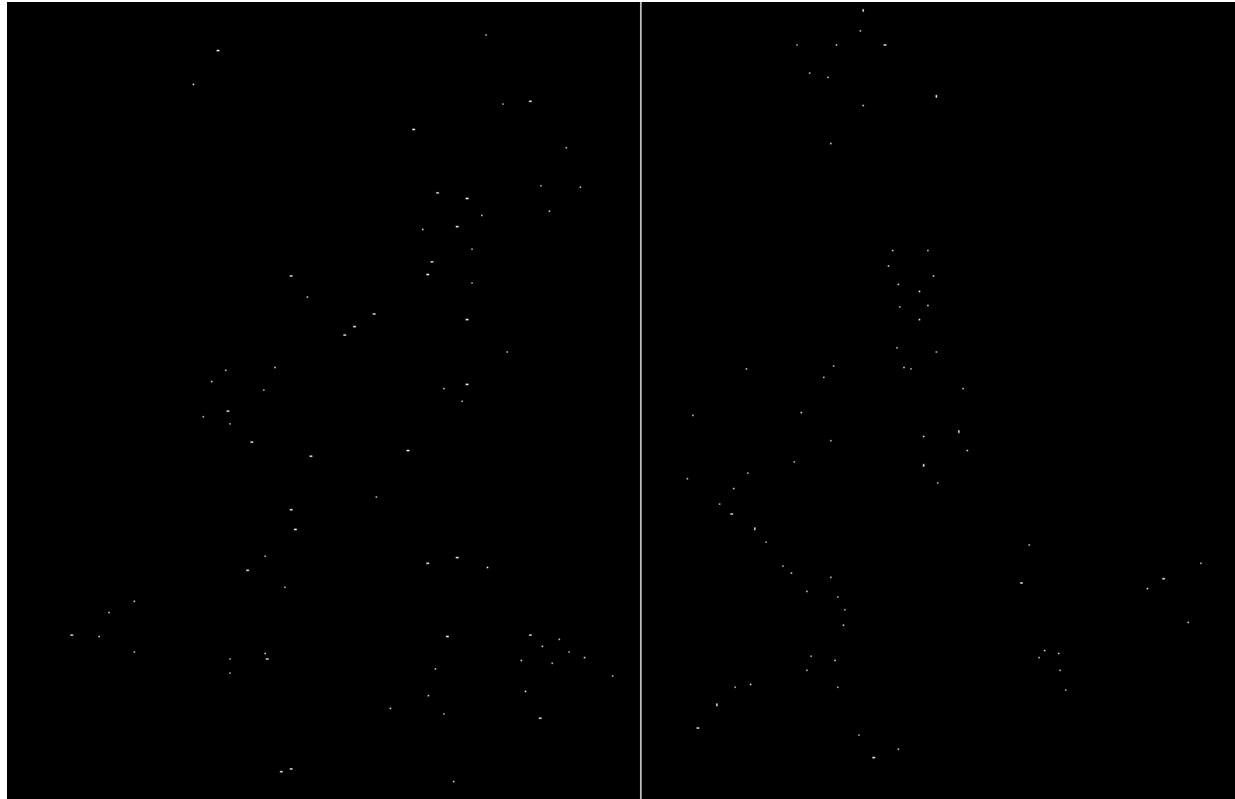


Image from slide
by S. Lazebnik

Harris Interest Point Detection

Why is the Harris algorithm shift and rotationally invariant

- **Shift invariant:** The convolutional Laplacian gradient kernel is applied across the image
 - Response of convolution only depend on gradient, not location
- **Rotationally invariant:** The eigenvalues are rotationally invariant
 - Eigenvectors project the gradient values to the principle axes
 - Eigenvalues remain unchanged
- **Not scale invariant:** The response of the Laplacian operator changes with local gradient and scale of the operator, σ

Gradients and Features – the HOG Algorithm

Gradients and Features

Gradients are powerful features

- Patterns of direction of maximum gradient can help identify objects in the image
- But, gradients are extremely noisy
- Idea, average histogram of gradients over patches on the image
- Basis of the Histogram of Oriented Gradients (HOG) algorithm, [Dalal and Triggs \(2005\)](#)
- HOGs generate powerful features
 - Particularly good for objects with long linear features
- Computationally intensive
- Typically apply HOG to each color channel or convert to gray-scale

Gradients and Features

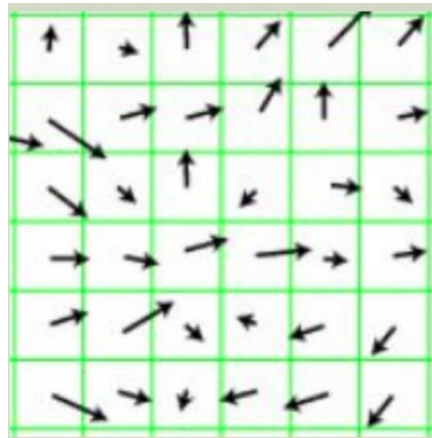
Many variations and details for HOG algorithms

1. Prepare the image
 - Resize
 - Local equalization and color gamma adjustments
 - Proper preparation required for consistent gradient estimation
2. Compute gradients around each pixel
3. Divide image into cells (often 8×8), weight gradients and compute histograms with 8 orientations
 - Log-polar coordinate weighting
4. Block normalize histograms (often 16×16 blocks)
 - Usually L1 or L2 norm

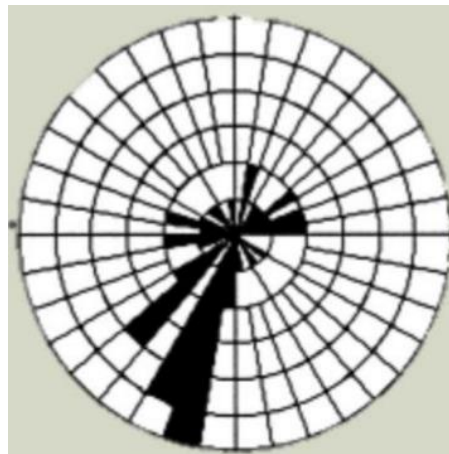
Result is $4 \times 8 = 32$ features per-block

Gradients and Features

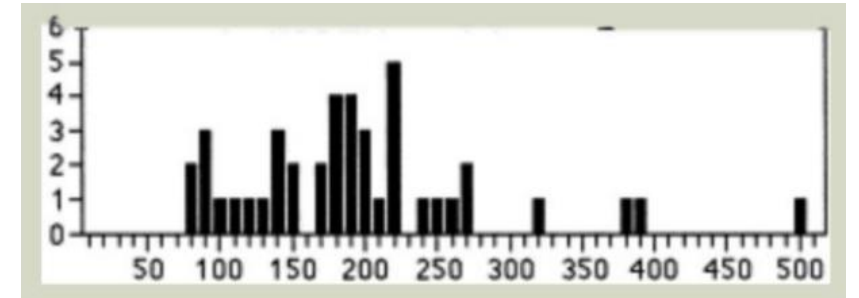
Basic steps of HOG algorithms



Gradients of
image patch



Histogram in
log-polar
coordinates

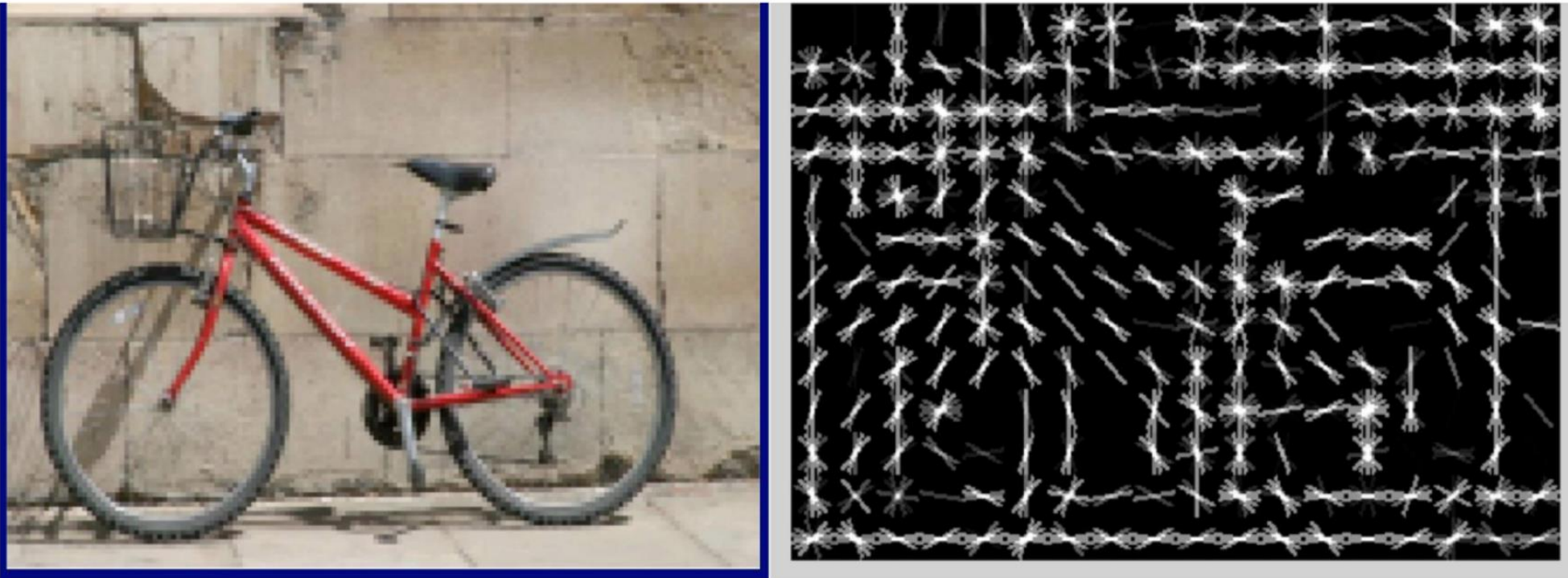


Normalized
feature vector

Adapted Stanford CS-131 slides Neibbles and Wu

Gradients and Features

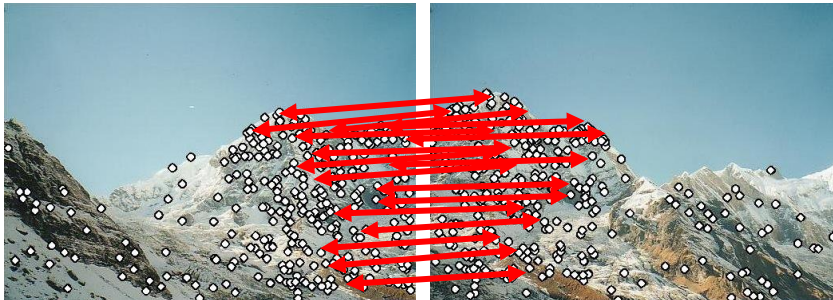
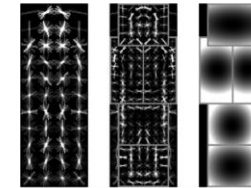
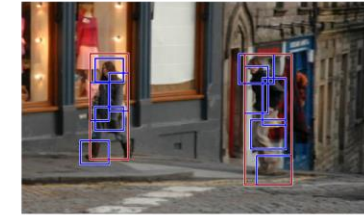
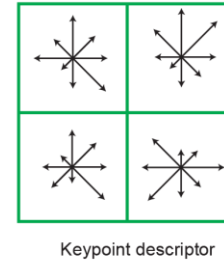
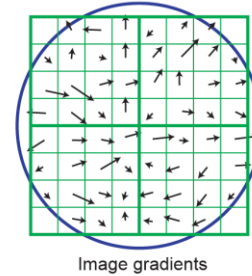
Example of HOG – features to identify a bicycle



Adapted Stanford CS-131 slides Neibbles and Wu

Features!

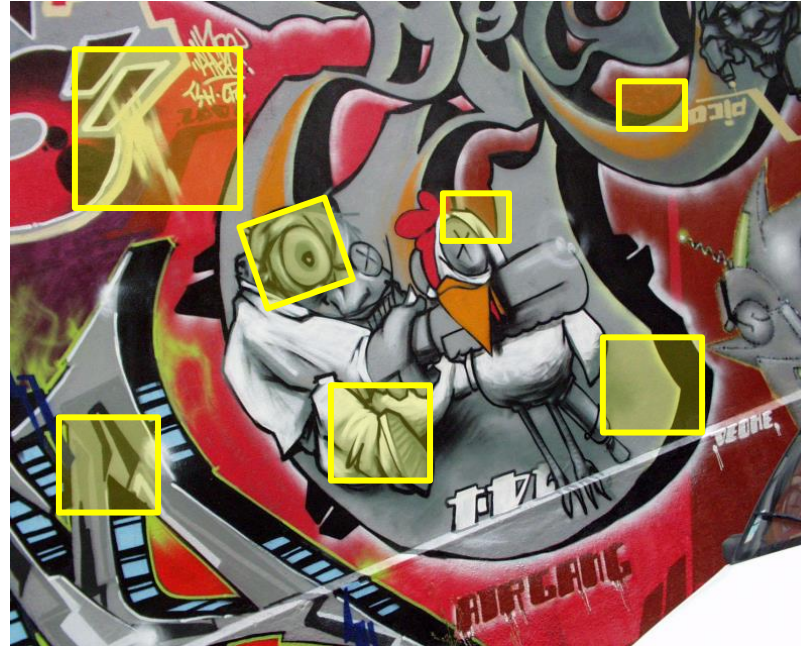
- Highly descriptive local regions
- Ways to describe those regions
- Useful for:
 - Matching
 - Recognition
 - Detection



Interest Point Descriptors

Interest Point Descriptors

Must identify interest points between complex images



Matching interest points can be difficult

Image from Oxford Dataset,
Mikolajczyk, et. al., 2005

Interest Point Descriptors

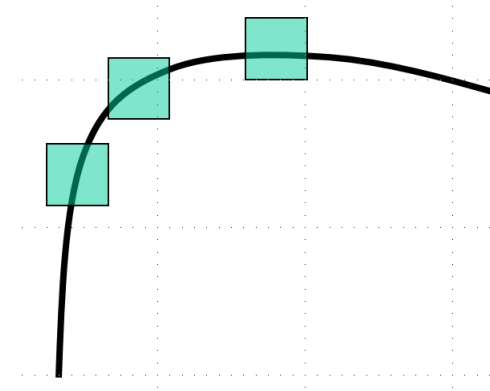
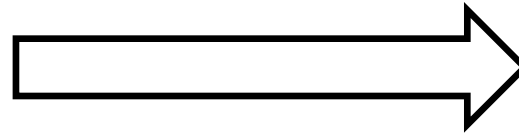
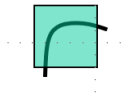
Must identify interest points between complex images

- Can we just use the set of interest points?
- Not directly, can't tell the features apart
- Need to use properties of the regions - **descriptors**
- Some ideas
 - Intensity
 - Color ratios
 - texture
 - Local gradient statistics ✓

Interest Point Descriptors

Must identify interest points between complex images

- Harris detector is not scale independent – not suitable



Harris detector
identifies this feature
as a corner

But, Harris detector
identifies this feature
as a series of edges

Interest Point Descriptors

Must identify interest points between complex images

- How well can intensity or color ratios identify features
- Create intensity histograms for region (patch) around feature
- Small shifts and rotations can change intensity and histogram
- Need a more **invariant** solution!

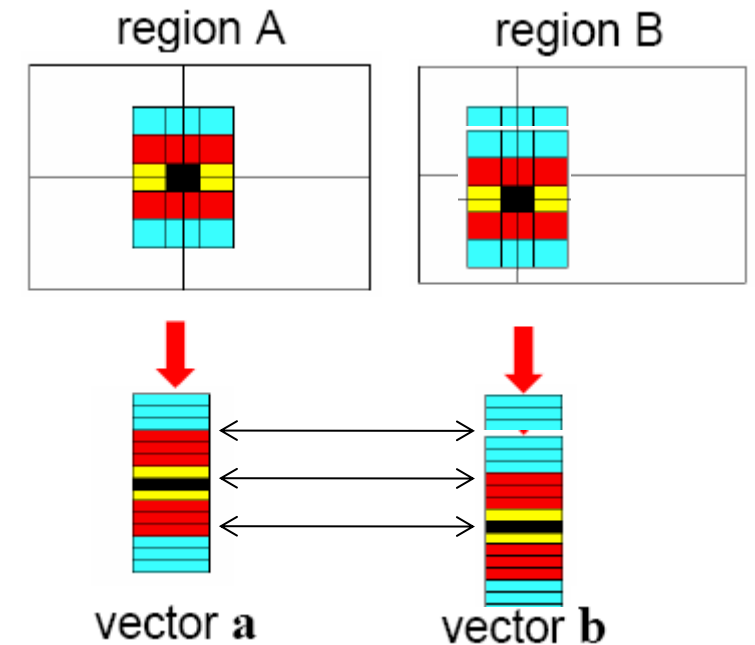


Figure by Andrew Zisserman

Interest Point Descriptors

Must uniquely identify interest points between complex images

- Need a more invariant solution!
- Scale invariant
- Shift invariant
- Rotationally invariant

Interest Point Descriptors

Key properties of good descriptors

- **Locality:** features describe parts and are robust to occlusion and clutter (no prior segmentation)
- **Distinctiveness:** features are identifiable from a large database of objects
- **Quantity:** many features can be generated for even small objects
- **Efficiency:** close to real-time performance
- **Extensibility:** can easily be extended to wide range of differing feature types, each adding robustness to changes

Interest Point Descriptors

Must identify interest points between complex images

- Many shift and rotationally invariant algorithms have been developed:
 - SHIFT
 - SURF
 - DAISY
 - BRIEF
 - CENTURE
 - ...
- Use hierarchy of scales with interest point descriptors

SHIFT Algorithm

The SIFT Algorithm

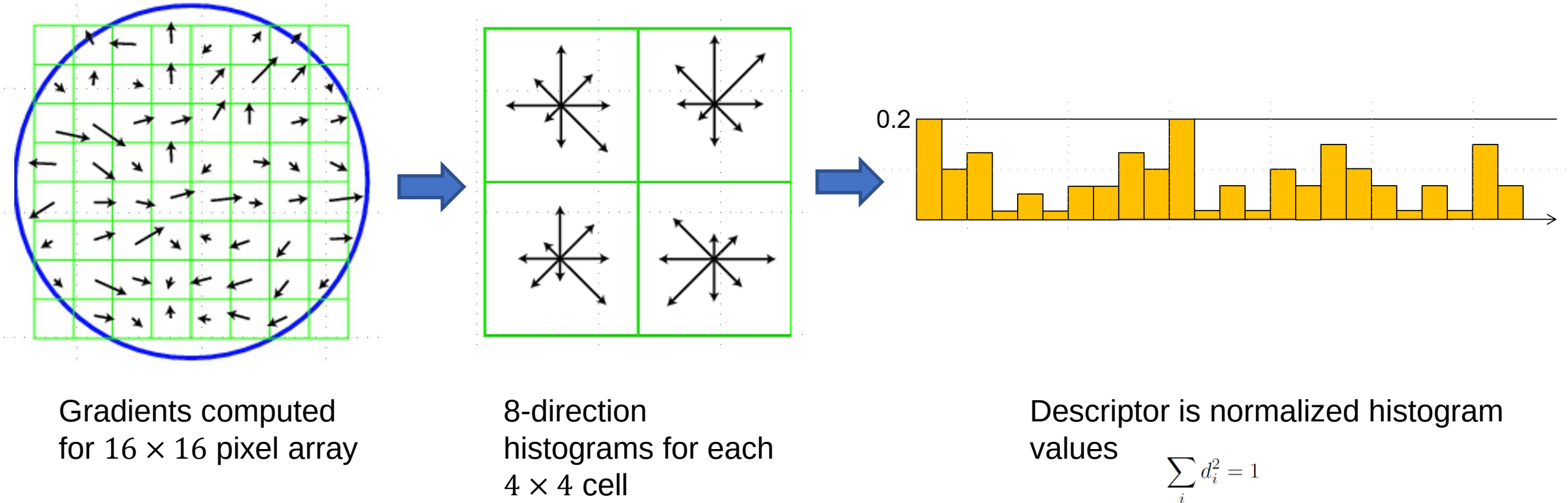
Scale Invariant Feature Transformation (SIFT)

- SIFT developed by [David Lowe \(1999\)](#) and [David Lowe \(2004\)](#)
- Algorithm
 1. Divide image into 16×16 patches
 2. Divide a 16×16 array of pixels into 16 4×4 cells
 3. Compute the gradients at each pixel
 4. Create a 8-direction Histogram Of Gradient (HOG) direction for each cell, gives $16 \times 8 = 128$ features
 5. Normalize the descriptor so that:

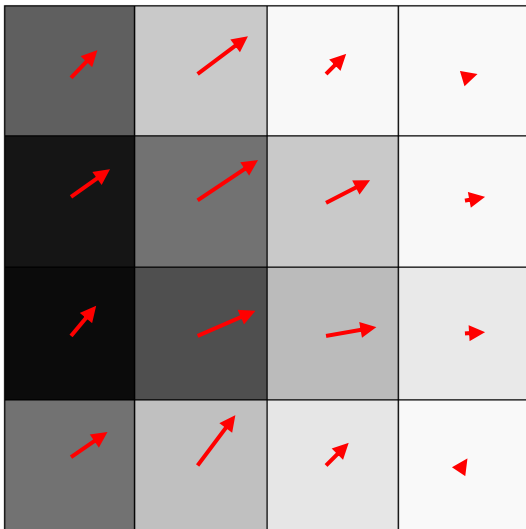
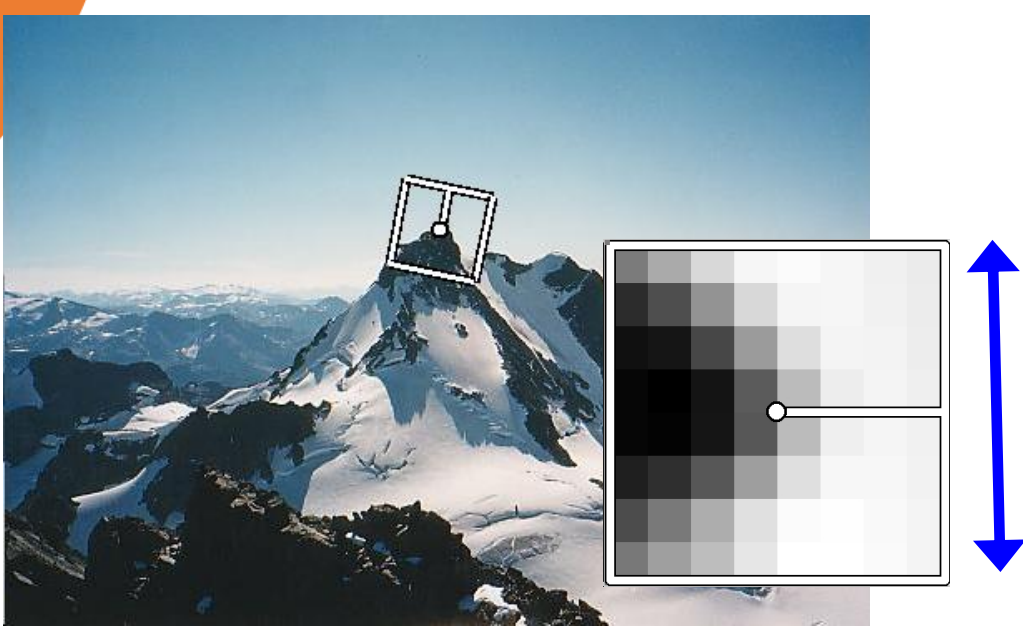
$$d_i < 0.2 \text{ and } \sum_i d_i^2 = 1.0$$

The SIFT Algorithm

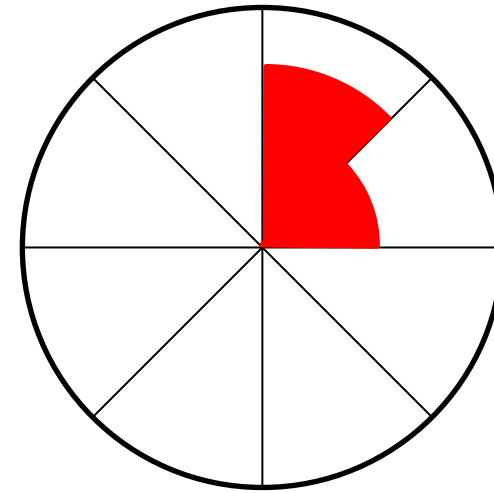
Scale Invariant Feature Transformation (SIFT)



Adapted from slide by David Lowe



Orientations in each of
the 16 pixels of the cell



The orientations all ended up in
two bins: 11 in one bin, 5 in the
other. (rough count)

5 11 0 0 0 0 0 0

Multi-Scale Interest Points

Multi-Scale Feature Identification

Pyramid structure enables extraction of interest points at several scales

- Pyramid scales from fine to coarse in octaves
- Features at each scale extracted
- Can use a variety of interest point descriptor algorithms
 - SIFT
 - Harris – Laplacian

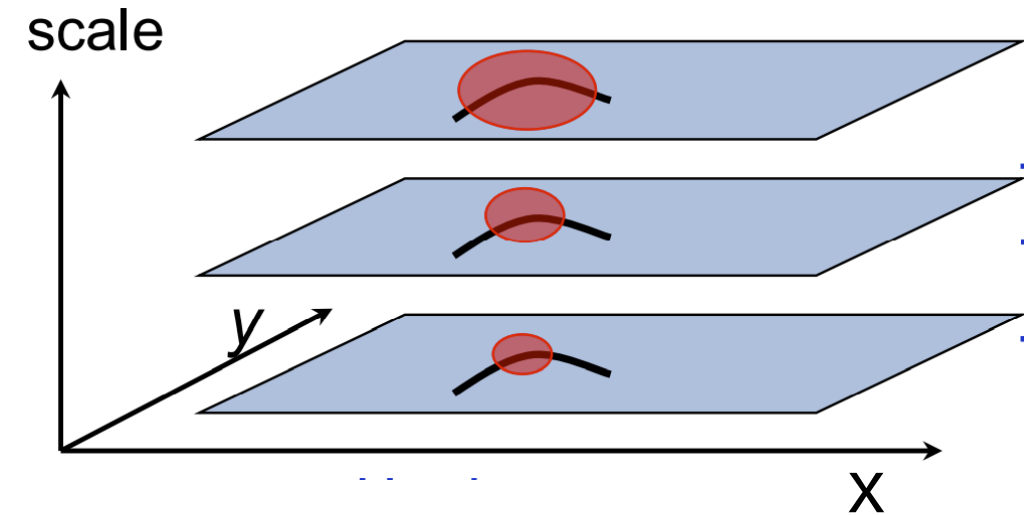


Diagram by Saad Bedros

Multi-Scale Feature Identification

Scaling using Laplacian or DoG

- Scaling applies convolution with kernel, K , on image I : $I_{scaled} = K * I$
- **Laplacian**

$$\mathcal{L} = \sigma^2 [G_{xx}(x, y, \sigma) + G_{yy}(x, y, \sigma)]$$

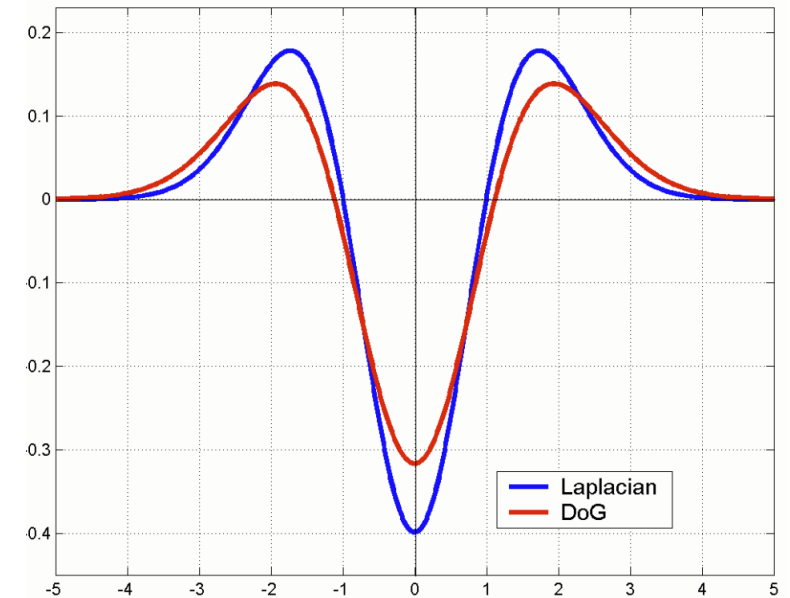
- **DoG**

$$DoG = G(x, y, k\sigma) - G(x, y, \sigma)$$

- Where Gaussian is:

$$G(x, y, \sigma) = \frac{1}{2\pi\sigma} \exp\left(-\frac{x^2 + y^2}{2\sigma^2}\right)$$

- Both kernels are rotationally invariant



Multi-Scale Feature Identification

Scaling DoG

- Scaling applies convolution with kernel, K , on image I : $I_{scaled} = K * I$
- Example: pyramid with DoG, $G(x, y, \sigma)$, $G(x, y, k\sigma)$, $G(x, y, k^2\sigma)$, ...

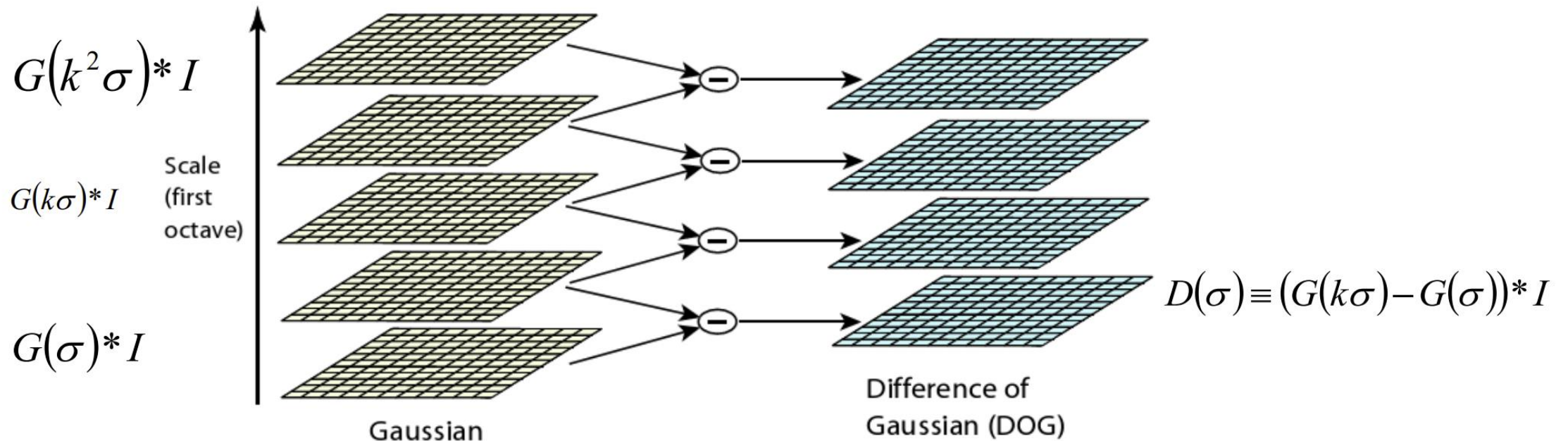
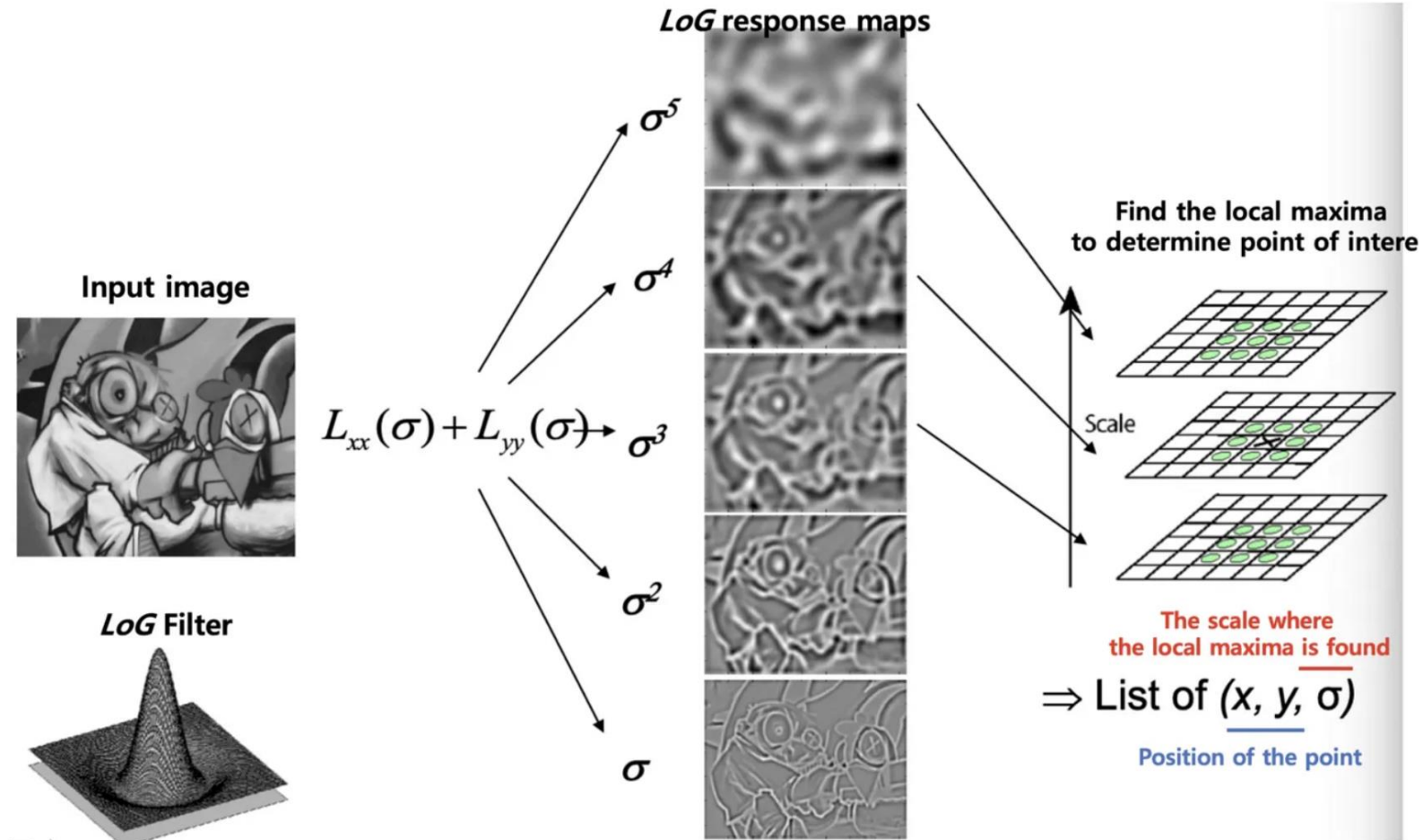


Diagram by Krystian
Mikolajczyk

Multi-Scale Feature Identification

Scaling DoG

- Scaling applies convolution with kernel using $k \sigma, k = 1, 2, \dots, 5$
- Determine scale with maximum of descriptor



Adapted from diagram by Krystian Mikolajczyk

Summary

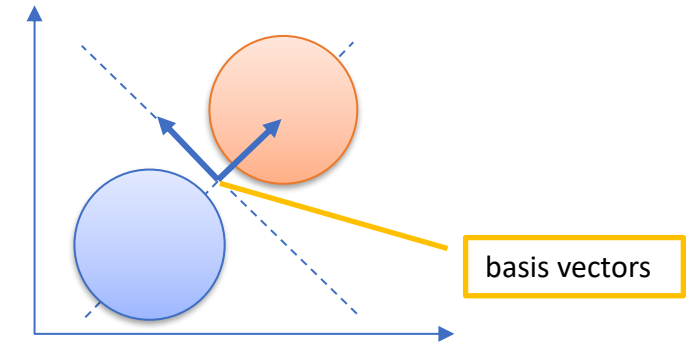
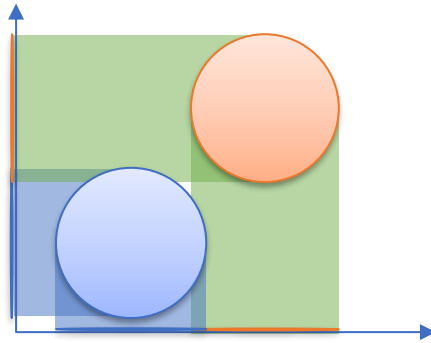
Key points for this lesson

- Use of gradients to detect edges
- How decomposition of gradients is used for general interest point detection
- How Harris corner detector works
- Using statistics of gradients are used to construct descriptors
- How to identify interest points with the SIFT algorithm
- How to detect interest points across scales

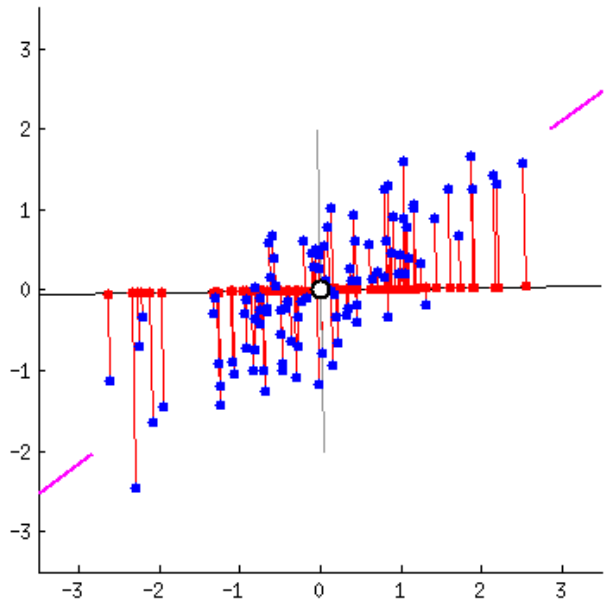
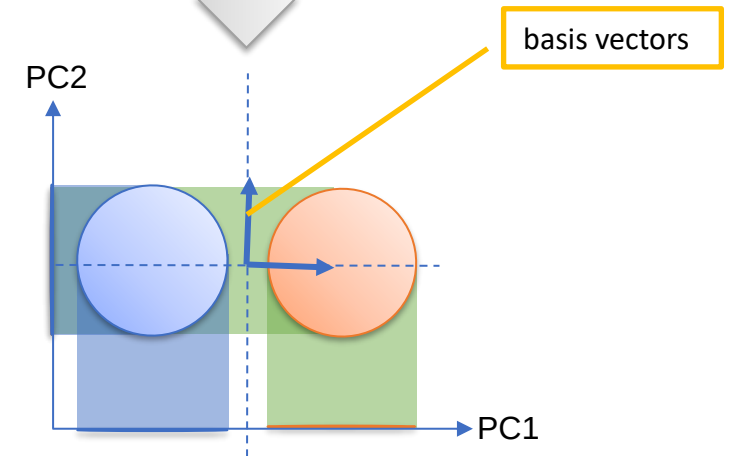


EIGENVECTOR AND EIGENVALUE

Dimension reduction with PCA



PCA



Dimension reduction

How to use PCA model?

s_1

Sepal length	Sepal width	Petal length	Petal width
5.1	3.5	1.4	0.2
4.9	3	1.4	0.2
4.7	3.2	1.3	0.2
4.6	3.1	1.5	0.2
5	3.6	1.4	0.2
5.4	3.9	1.7	0.4
4.6	3.4	1.4	0.3
5	3.4	1.5	0.2
4.4	2.9	1.4	0.2
4.9	3.1	1.5	0.1
5.4	3.7	1.5	0.2
4.8	3.4	1.6	0.2
4.8	3	1.4	0.1
4.3	3	1.1	0.1
5.8	4	1.2	0.2
5.7	4.4	1.5	0.4
5.4	3.9	1.3	0.4
5.1	3.5	1.4	0.3
5.7	3.8	1.7	0.3
5.1	3.8	1.5	0.3
5.4	3.4	1.7	0.2
5.1	3.7	1.5	0.4
:	:	:	:

150x4

mean

5.84 3.05 3.76 1.20

PCA model

Eigen vectors

u_1

0.36	0.66	-0.58	0.32
-0.08	0.73	0.60	-0.32
0.86	-0.18	0.07	-0.48
0.36	-0.07	0.55	0.75

U
4x4

Eigen values

λ_1

4.22	0.24	0.08	0.02
------	------	------	------

λ

$s_1 \quad m \quad u_1$

$$(5.1-5.84)*(0.36)+$$

$$(3.5-3.05)*(-0.08)+$$

$$(1.4-3.76)*(0.86)+$$

$$(0.2-1.20)*(0.36) = -2.69$$

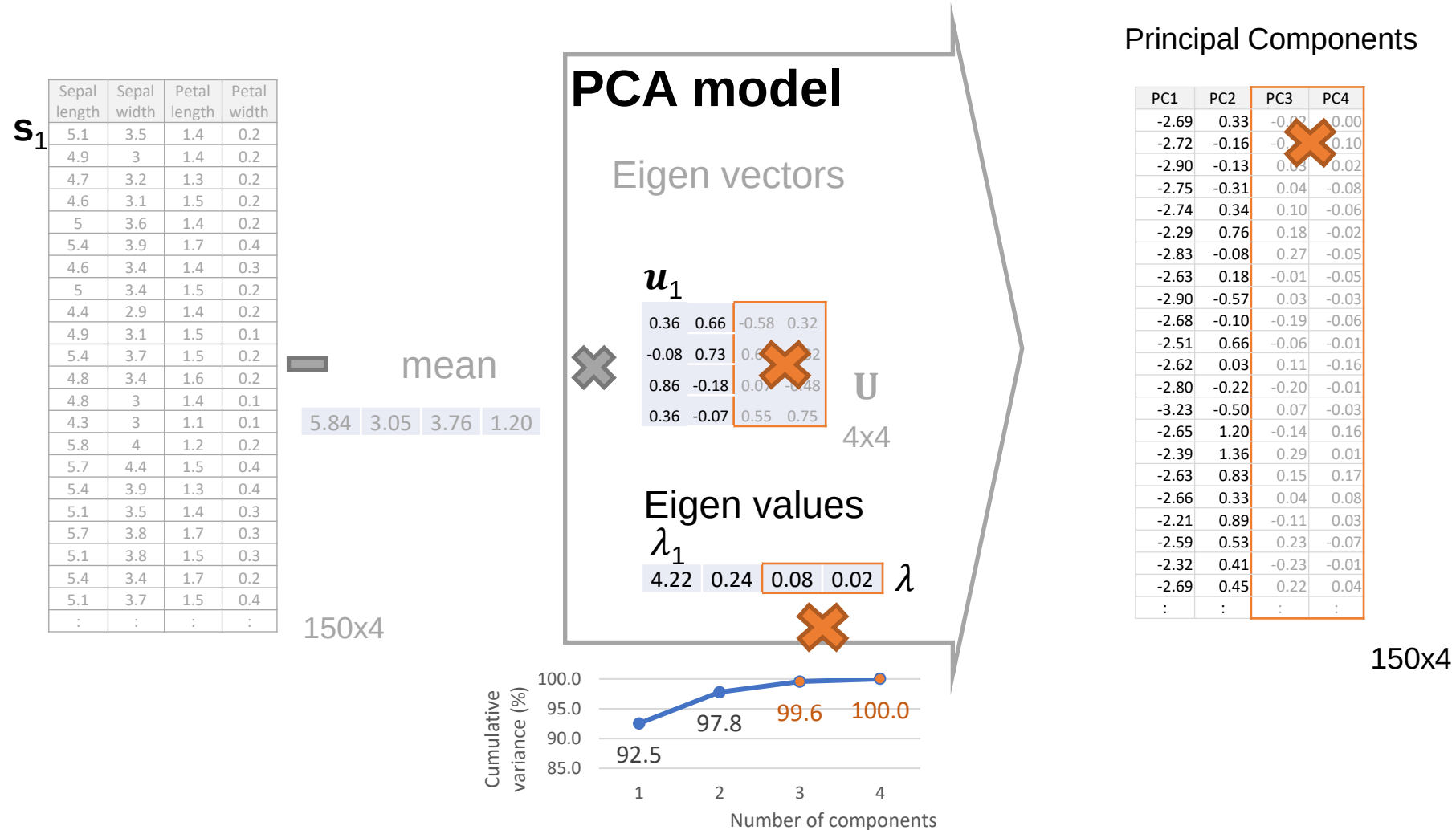
Principal Components

PC1	PC2	PC3	PC4
-2.69	0.33	-0.02	0.00
-2.72	-0.16	-0.20	0.10
-2.90	-0.13	0.03	0.02
-2.75	-0.31	0.04	-0.08
-2.74	0.34	0.10	-0.06
-2.29	0.76	0.18	-0.02
-2.83	-0.08	0.27	-0.05
-2.63	0.18	-0.01	-0.05
-2.90	-0.57	0.03	-0.03
-2.68	-0.10	-0.19	-0.06
-2.51	0.66	-0.06	-0.01
-2.62	0.03	0.11	-0.16
-2.80	-0.22	-0.20	-0.01
-3.23	-0.50	0.07	-0.03
-2.65	1.20	-0.14	0.16
-2.39	1.36	0.29	0.01
-2.63	0.83	0.15	0.17
-2.66	0.33	0.04	0.08
-2.21	0.89	-0.11	0.03
-2.59	0.53	0.23	-0.07
-2.32	0.41	-0.23	-0.01
-2.69	0.45	0.22	0.04
:	:	:	:

150x4

Dimension reduction

How many PCA components to choose?



2D pairwise plot of the Principal Component

